

CMD Engine

- [1. CMD Engine – Kompakte System-Referenz für KI-Modelle](#)
- [2. Operationen](#)
- [3. EXEC – Kommando-Ausführung](#)
 - [3.1 Parameter](#)
 - [3.2 Automatische Exit-Code-Prüfung](#)
 - [Erwarteter Non-Zero Exit-Code](#)
 - [Automatische Prüfung deaktivieren](#)
 - [3.3 Timeout](#)
 - [3.4 outputFile](#)
 - [3.5 Arbeitsverzeichnis \(WORKDIR\)](#)
 - [Absoluter Skript- oder Programmpfad](#)
 - [Systembefehle und relative Kommandos](#)
 - [3.6 Relative outputFile-Pfade](#)
- [4. ASSERT – Validierung](#)
 - [EXITCODE](#)
 - [CONTAINS](#)
 - [NOT_CONTAINS](#)
 - [COUNT](#)
- [5. BUFFER – Datenextraktion](#)
 - [5.1 BUFFER Actions](#)
 - [TEXT](#)
 - [FILTER](#)
 - [LINE](#)
 - [LAST_LINE](#)
 - [COUNT](#)
- [6. Verwendung von BUFFER-Werten](#)
- [7. Vollständiges Beispiel](#)
- [8. Troubleshooting & Best Practices](#)
 - [Unerwarteter Exit-Code](#)
 - [ASSERT schlägt fehl](#)
 - [Condition wird übersprungen](#)
 - [Große Outputs](#)
- [9. Kurzreferenz](#)

Die CMD-Engine ermöglicht die Ausführung lokaler Windows-Kommandos und Skripte. Sie unterstützt die automatische Prüfung von Exit-Codes, Timeouts, das Speichern von Kommandoausgaben in Dateien sowie die Weiterverarbeitung und Validierung des Outputs über **BUFFER** und **ASSERT**.

1. CMD Engine – Kompakte System-Referenz für KI-Modelle

```
1 # AGATE CMD-Engine Syntax Referenz
2 # Nutze dies für Testfall-Generierung und Analyse
3
4 # EXEC
5 - type: CMD
6   op: EXEC
```

```

7  command: "...
8  response: "...          # Optional: Name des gespeicherten
  CommandResult
9  expectedExitCode: 0      # Optional, Default: 0
10 checkExitCode: true     # Optional, Default: true
11 timeout: 30             # Optional, Sekunden, Default: 30
12 outputFile: "...       # Optional: stdout/stderr zusätzlich
  lokal speichern
13 condition: "...        # Optional
14
15 # ASSERT
16 - type: CMD
17   op: ASSERT
18   response: "...
19   action: <EXITCODE|CONTAINS|NOT_CONTAINS|EQUALS|NOT_EQUALS|COUNT>
20   value: "...
21   expected: "...
22   condition: "...
23
24 # BUFFER
25 - type: CMD
26   op: BUFFER
27   response: "...
28   action: <TEXT|FILTER|LINE|LAST_LINE|COUNT>
29   value: "...
30   name: "...
31   condition: "...
32

```

2. Operationen

Die CMD-Engine unterstützt drei Operationen:

Operation	Beschreibung
EXEC	Führt einen lokalen Windows-Systembefehl oder ein Skript aus.
BUFFER	Extrahiert Informationen aus dem Output eines vorherigen EXEC .
ASSERT	Validiert Exit-Code oder Output eines vorherigen EXEC bzw. BUFFER .

3. EXEC – Kommando-Ausführung

EXEC führt einen lokalen System- oder CLI-Befehl aus.

Die Engine erfasst dabei:

- Exit-Code

- stdout/stderr
- Timeout-Status
- Ausführungsdauer

Das Ergebnis kann über `response` gespeichert und anschließend mit `BUFFER` oder `ASSERT` weiterverarbeitet werden.

3.1 Parameter

Parameter	Beschreibung	Erforderlich	Default
<code>command</code>	Auszuführender Windows-, CLI- oder Skript-Befehl	Ja	–
<code>response</code>	Name, unter dem das Kommandoergebnis gespeichert wird	Nein	–
<code>expectedExitCode</code>	Erwarteter Exit-Code	Nein	<code>0</code>
<code>checkExitCode</code>	Automatische Prüfung des Exit-Codes aktivieren/deaktivieren	Nein	<code>true</code>
<code>timeout</code>	Maximale Ausführungszeit in Sekunden	Nein	<code>30</code>
<code>outputFile</code>	Speichert den Kommando-Output zusätzlich in einer lokalen Datei	Nein	–

condition	Bedingung für die Ausführung des Schritts	Nein	–
-----------	---	------	---

Einfaches Beispiel:

```

1 - type: CMD
2   op: EXEC
3   command: "java --version"
4   response: java_version
5

```

3.2 Automatische Exit-Code-Prüfung

Standardmäßig erwartet die CMD-Engine:

```

1 expectedExitCode: 0
2 checkExitCode: true
3

```

Damit führt beispielsweise ein Kommando mit Exit-Code **1** automatisch zu einem fehlgeschlagenen Testschritt.

```

1 - type: CMD
2   op: EXEC
3   command: "java --version"
4

```

entspricht funktional:

```

1 - type: CMD
2   op: EXEC
3   command: "java --version"
4   expectedExitCode: 0
5   checkExitCode: true
6

```

Erwarteter Non-Zero Exit-Code

Ein Non-Zero Exit-Code kann ein fachlich erwartetes Ergebnis sein:

```

1 - type: CMD
2   op: EXEC
3   command: "exit /b 7"
4   response: exit_result
5   expectedExitCode: 7
6

```

Der Schritt ist erfolgreich, wenn das Kommando tatsächlich Exit-Code **7** liefert.

Automatische Prüfung deaktivieren

Für Negativtests kann die automatische Prüfung deaktiviert werden:

```
1 - type: CMD
2   op: EXEC
3   command: "exit /b 5"
4   response: failed_command
5   checkExitCode: false
6
```

Der Exit-Code wird weiterhin gespeichert und kann anschließend explizit geprüft werden:

```
1 - type: CMD
2   op: ASSERT
3   response: failed_command
4   action: EXITCODE
5   expected: 5
6
```

Damit sind zwei Varianten möglich:

```
1 EXEC + expectedExitCode
2
```

für eine direkte Validierung und

```
1 EXEC + checkExitCode:false
2 ASSERT EXITCODE
3
```

für Negativtests oder eine spätere explizite Validierung.

3.3 Timeout

Die maximale Ausführungsdauer kann über `timeout` definiert werden.

Der Wert wird in Sekunden angegeben.

```
1 - type: CMD
2   op: EXEC
3   command: "ping 127.0.0.1 -n 10 > nul"
4   timeout: 1
5
```

Wird das Timeout überschritten, schlägt der Schritt fehl.

Wenn kein `timeout` angegeben wird, gilt:

```
1 30 Sekunden
2
```

3.4 outputFile

Mit `outputFile` kann der komplette Kommando-Output zusätzlich als lokale Datei gespeichert werden.

```
1 - type: CMD
2   op: EXEC
3   command: "java --version"
```

```
4 response: java_version
5 outputFile: "output/java-version.txt"
6
```

Der Output steht weiterhin über `response` für `BUFFER` und `ASSERT` zur Verfügung.
`outputFile` ist somit eine zusätzliche Persistierung und ersetzt `response` nicht.

3.5 Arbeitsverzeichnis (WORKDIR)

Die CMD-Engine bestimmt das Arbeitsverzeichnis abhängig vom ausgeführten Kommando.

Absoluter Skript- oder Programmpfad

Bei einem absoluten Windows-Pfad, beispielsweise:

```
1 - type: CMD
2   op: EXEC
3   command: "C:\\Services\\App\\run.bat"
4
```

wird das übergeordnete Verzeichnis automatisch als WORKDIR verwendet:

```
1 C:\\Services\\App
2
```

Dadurch können Skripte lokale Konfigurationsdateien, Libraries oder andere relative Ressourcen in ihrem eigenen Verzeichnis verwenden.

Systembefehle und relative Kommandos

Bei Kommandos wie:

```
1 command: "java --version"
2
```

oder:

```
1 command: "dir"
2
```

wird kein separates WORKDIR gesetzt.

Das Kommando wird im Arbeitsverzeichnis des laufenden AGATE-Prozesses ausgeführt.

3.6 Relative outputFile-Pfade

Ein relativer `outputFile` -Pfad orientiert sich am effektiven Arbeitsverzeichnis des CMD-Schritts.

Beispiel mit absolutem Skript:

```
1 - type: CMD
2   op: EXEC
```

```
3 command: "C:\\Services\\App\\run.bat"
4 outputFile: "output/result.txt"
5
```

Ergebnis:

```
1 C:\Services\App\output\result.txt
2
```

Bei einem Systemkommando:

```
1 - type: CMD
2   op: EXEC
3   command: "java --version"
4   outputFile: "output/java-version.txt"
5
```

liegt die Datei relativ zum Arbeitsverzeichnis des AGATE-Prozesses.

4. ASSERT – Validierung

ASSERT prüft den Exit-Code oder Inhalt eines zuvor gespeicherten Ergebnisses.

Schlägt die Prüfung fehl, schlägt auch der Testschritt fehl.

Action	Beschreibung	Parameter
EXITCODE	Prüft den Exit-Code	expected
CONTAINS	Prüft, ob ein Text enthalten ist	value
NOT_CONTAINS	Prüft, ob ein Text nicht enthalten ist	value
EQUALS	Prüft auf exakte Übereinstimmung nach trim()	value
NOT_EQUALS	Prüft auf Abweichung nach trim()	value
COUNT	Prüft die Anzahl eines Suchbegriffs	value , expected

EXITCODE

```
1 - type: CMD
2   op: ASSERT
3   response: java_version
```

```
4  action: EXITCODE
5  expected: 0
6
```

CONTAINS

```
1  - type: CMD
2    op: ASSERT
3    response: java_version
4    action: CONTAINS
5    value: "OpenJDK"
6
```

NOT_CONTAINS

```
1  - type: CMD
2    op: ASSERT
3    response: java_version
4    action: NOT_CONTAINS
5    value: "ERROR"
6
```

COUNT

```
1  - type: CMD
2    op: ASSERT
3    response: java_version
4    action: COUNT
5    value: "OpenJDK"
6    expected: 2
7
```

5. BUFFER – Datenextraktion

BUFFER extrahiert Informationen aus dem Output eines vorherigen **EXEC** und speichert das Ergebnis unter **name**.

```
1  - type: CMD
2    op: BUFFER
3    response: java_version
4    action: TEXT
5    name: java_version_text
6
```

Der erzeugte Wert kann anschließend über:

```
1  {B[java_version_text]}
2
```

verwendet werden.

Wichtig: Für **name** sollte ein anderer Bezeichner als für **response** verwendet werden. Dadurch bleibt das ursprüngliche Kommandoergebnis einschließlich Exit-Code erhalten.

5.1 BUFFER Actions

Action	Beschreibung	Parameter
TEXT	Speichert den kompletten getrimmten Output	–
FILTER	Speichert nur Zeilen, die einen Suchbegriff enthalten	value
LINE	Extrahiert eine Zeile; 0 = erste Zeile	value optional, Default 0
LAST_LINE	Extrahiert relativ vom Ende; 0 = letzte Zeile	value optional, Default 0
COUNT	Zählt das Vorkommen eines Suchbegriffs	value

TEXT

```
1 - type: CMD
2   op: BUFFER
3   response: java_version
4   action: TEXT
5   name: complete_output
6
```

FILTER

```
1 - type: CMD
2   op: BUFFER
3   response: java_version
4   action: FILTER
5   value: "Runtime"
6   name: runtime_line
7
```

LINE

```
1 - type: CMD
2   op: BUFFER
3   response: java_version
4   action: LINE
5   value: 0
6   name: first_line
7
```

LAST_LINE

```
1 - type: CMD
```

```
2 op: BUFFER
3 response: java_version
4 action: LAST_LINE
5 value: 0
6 name: last_line
7
```

COUNT

```
1 - type: CMD
2 op: BUFFER
3 response: java_version
4 action: COUNT
5 value: "OpenJDK"
6 name: openjdk_count
7
```

6. Verwendung von BUFFER-Werten

Ein BUFFER-Ergebnis kann in späteren Schritten über `{B[...]}` verwendet werden.

```
1 - type: CMD
2 op: EXEC
3 command: "echo %OS%"
4 response: os_result
5
6 - type: CMD
7 op: BUFFER
8 response: os_result
9 action: TEXT
10 name: os_version
11
12 - type: CMD
13 condition: "'{B[os_version]}' == 'Windows_NT'"
14 op: EXEC
15 command: "ver"
16 response: windows_version
17
```

7. Vollständiges Beispiel

```
1 testCases:
2
3 - id: TC_CMD_Engine_Demo
4   description: "CMD Engine - Full Demo"
5   stage: "*"
6   priority: HIGH
7
8   steps:
9
10    # Standard EXEC
11    - type: CMD
12      op: EXEC
13      command: "java --version"
14      response: java_version
15
16    # Output prüfen
17    - type: CMD
18      op: ASSERT
19      response: java_version
20      action: CONTAINS
```

```

21     value: "openjdk"
22
23     # Erste Zeile extrahieren
24     - type: CMD
25       op: BUFFER
26       response: java_version
27       action: LINE
28       value: 0
29       name: java_first_line
30
31     # Expliziter erwarteter Exit-Code
32     - type: CMD
33       op: EXEC
34       command: "exit /b 7"
35       response: exit_7
36       expectedExitCode: 7
37
38     # Exit-Code zusätzlich prüfen
39     - type: CMD
40       op: ASSERT
41       response: exit_7
42       action: EXITCODE
43       expected: 7
44
45     # Automatische Exit-Code-Prüfung deaktivieren
46     - type: CMD
47       op: EXEC
48       command: "exit /b 5"
49       response: exit_5
50       checkExitCode: false
51
52     - type: CMD
53       op: ASSERT
54       response: exit_5
55       action: EXITCODE
56       expected: 5
57
58     # Timeout explizit definieren
59     - type: CMD
60       op: EXEC
61       command: "echo timeout test"
62       response: timeout_test
63       timeout: 5
64
65     # Output zusätzlich in Datei schreiben
66     - type: CMD
67       op: EXEC
68       command: "echo AGATE CMD outputFile test"
69       response: output_file_test
70       outputFile: "output/cmd-output.txt"
71
72     - type: CMD
73       op: ASSERT
74       response: output_file_test
75       action: CONTAINS
76       value: "AGATE CMD outputFile test"
77

```

8. Troubleshooting & Best Practices

Unerwarteter Exit-Code

Da `checkExitCode` standardmäßig `true` ist, führt ein unerwarteter Non-Zero Exit-Code automatisch zu einem fehlgeschlagenen Schritt.

Wenn ein Non-Zero Exit-Code erwartet wird, sollte entweder:

```
1 expectedExitCode: 7
2
```

gesetzt oder für Negativtests:

```
1 checkExitCode: false
2
```

verwendet werden.

ASSERT schlägt fehl

Prüfe den tatsächlichen Output des vorherigen `EXEC`.

CLI-Programme liefern häufig zusätzliche Leerzeichen, Zeilenumbrüche oder mehrere Output-Zeilen.

Mit `BUFFER` können relevante Teile gezielt extrahiert werden.

Condition wird übersprungen

Bei String-Vergleichen sollten die aufgelösten Werte in einfachen Anführungszeichen stehen:

```
1 condition: "'{B[my_var]}' == 'value'"
2
```

Große Outputs

Der Output eines CMD-Kommandos wird für die weitere Verarbeitung im Speicher gehalten.

Bei sehr großen Outputs sollte möglichst früh mit `BUFFER FILTER`, `LINE` oder einer anderen gezielten Extraktion gearbeitet werden.

Wenn der vollständige Output dauerhaft benötigt wird, kann zusätzlich `outputFile` verwendet werden.

9. Kurzreferenz

```
1 CMD
2   | EXEC
3   |   | command
4   |   | response
5   |   | expectedExitCode [default: 0]
6   |   | checkExitCode   [default: true]
7   |   | timeout         [default: 30s]
8   |   | outputFile
9
10  | BUFFER
11  |   | TEXT
```

12			FILTER
13			LINE
14			LAST_LINE
15			COUNT
16			
17			ASSERT
18			EXITCODE
19			CONTAINS
20			NOT_CONTAINS
21			EQUALS
22			NOT_EQUALS
23			COUNT
24			